

Práctica 0: Familiarización con el escenario CasaRobot

Esta práctica no introduce ningún algoritmo nuevo de Inteligencia Artificial. Su único objetivo es que te familiarices con `HouseWorld` y `Robot` (archivo `common/house_world.py`), el escenario común que usarán las cuatro prácticas de la asignatura, escribiendo un puñado de funciones pequeñas que usan su API.

La Sección 2 de este documento es, además, la **referencia completa de esa API**: consúltala siempre que la necesites en las Prácticas 1 a 4, no hace falta releer el código fuente.

1. Entrega

La entrega se realizará en el aula, durante la clase de prácticas del 22 de septiembre de 2026, a través de la tarea habilitada para ello en el aula virtual.

A diferencia de las Prácticas 1 a 4, esta Práctica 0 **no** tiene cuestionario de defensa: es material de preparación, no puntúa contenido de IA propiamente dicho.

2. Infraestructura proporcionada

Todo lo de esta sección ya está implementado en `common/house_world.py` y es común a las 4 prácticas: no se edita en ninguna de ellas.

2.1. Convención de coordenadas

Las coordenadas son **1-based**: $x \in [1, \text{width}]$, $y \in [1, \text{height}]$. El origen $(1, 1)$ está arriba a la izquierda; y crece hacia abajo (igual que las filas de una terminal). El borde exterior completo ($x = 1$, $x = \text{width}$, $y = 1$, $y = \text{height}$) es siempre muro.

2.2. RoomType y OBJECT_CATALOG

`RoomType` es un Enum con 4 valores: `KITCHEN` (“Cocina”), `BEDROOM` (“Dormitorio”), `BATHROOM` (“Baño”) y `LIVING_ROOM` (“Salón”). `OBJECT_CATALOG` es un diccionario que asocia cada `RoomType` con una tupla de nombres de objeto, y que usa el generador para poblar cada habitación. Por ejemplo, para `RoomType.KITCHEN` la tupla es `("nevera", "fogón", "fregadero", "microondas")`.

2.3. DIRECTIONS

`DIRECTIONS` es un diccionario que asocia cada acción cardinal con su desplazamiento $(\Delta x, \Delta y)$:

- `"N"` $\rightarrow (0, -1)$
- `"S"` $\rightarrow (0, +1)$
- `"E"` $\rightarrow (+1, 0)$
- `"W"` $\rightarrow (-1, 0)$

`Robot.move` acepta además la acción especial `"STAY"` (no está en `DIRECTIONS`), que nunca falla y no desplaza al robot.

2.4. Clase Room

Room es un dataclass inmutable con los campos `room_id` (int), `room_type` (RoomType) y `x0`, `y0`, `x1`, `y1` (int; esquinas 1-based e inclusivas). Métodos:

- `contains(x, y) ->bool`: si (x, y) está dentro del rectángulo de la habitación.
- `center() ->(int, int)`: celda central (división entera).

2.5. Clase HouseWorld

2.5.1. Constructor

```
HouseWorld(width, height, *, num_rooms=5, min_room_size=3,
            max_room_size=6, object_density=0.7, seed=None)
```

- `width, height`: tamaño de la rejilla. Mínimo 7×7 .
- `num_rooms`: número de habitaciones a intentar colocar (≥ 1). Si no caben todas por falta de espacio, se generan menos sin lanzar ningún error (comprobar `len(world.rooms)` si importa).
- `min_room_size, max_room_size`: ancho y alto de cada habitación, sorteados independientemente entre `min_room_size` y `max_room_size` (ambos incluidos). Debe cumplirse $\text{max_room_size} \leq \min(\text{width}, \text{height}) - 3$.
- `object_density` $\in [0, 1]$: probabilidad de que cada objeto típico de una habitación (según `OBJECT_CATALOG`) llegue a colocarse en ella.
- `seed`: semilla aleatoria. Los mismos parámetros y la misma semilla generan siempre la misma casa.

Lanza `ValueError` si los parámetros son inconsistentes (tamaños demasiado pequeños o demasiado grandes para la rejilla, `num_rooms` < 1 , densidad fuera de $[0, 1]$).

2.5.2. Propiedades

<code>width, height</code>	<code>int</code> — dimensiones de la casa
<code>rooms</code>	<code>List[Room]</code> — copia de la lista de habitaciones

2.5.3. API de consulta

Estos métodos no modifican nada: sirven para *planificar* (son los que se usarán para formular problemas de búsqueda, bases de conocimiento lógicas, etc. en las Prácticas 1 a 4).

<code>in_bounds(x, y) ->bool</code>	si (x, y) está dentro de la rejilla
<code>is_wall(x, y) ->bool</code>	<code>True</code> si es muro o está fuera de la rejilla
<code>is_free(x, y) ->bool</code>	lo contrario de <code>is_wall</code> , y dentro de la rejilla
<code>neighbors(x, y)</code> <code>->List[(int,int)]</code>	vecinos en las 4 direcciones cardinales que son libres
<code>room_at(x, y) ->Room None</code>	la habitación que contiene (x, y) ; <code>None</code> si es pasillo o muro
<code>objects_at(x, y) ->List[str]</code>	nombres de los objetos en esa celda (lista vacía si no hay)
<code>all_objects()</code> <code>->List[((int,int), str)]</code>	todos los objetos de la casa, ordenados por (y, x)
<code>random_free_cell() ->(int,int)</code>	una celda libre al azar (usa la semilla del constructor)

2.5.4. Visualización: render, print, legend

`print(**kwargs)` llama a `render(**kwargs)` e imprime el resultado; ambos aceptan los mismos parámetros:

Parámetro	Por defecto	Significado
<code>robot_pos</code>	<code>None</code>	celda (x,y) donde dibujar el robot
<code>path</code>	<code>None</code>	secuencia de celdas a resaltar como ruta (color distintivo + *)
<code>heat</code>	<code>None</code>	{(x,y): valor}; mapa de calor rojo (bajo) → verde (alto) con el valor superpuesto
<code>precision</code>	3	decimales al formatear los valores de <code>heat</code>
<code>cell_width</code>	3	ancho de cada celda, en caracteres
<code>cell_height</code>	1	alto de cada celda, en filas de terminal
<code>use_color</code>	<code>True</code>	color ANSI de 24 bits; <code>False</code> = ASCII
<code>show_axes</code>	<code>True</code>	cabecera de columnas y etiquetas y=N

Prioridad cuando varias capas coinciden en una celda: robot > muro > heat > path > contenido normal (habitación/pasillo/objeto). Si el texto de una celda no cabe en `cell_width`, se muestra un relleno "###" en vez de truncarlo: truncar "0.125" a "0.1" lo haría parecer un valor distinto en vez de un problema de formato.

`legend(*, use_color=True, show_path=False, show_heat=False) ->str` devuelve el texto que explica los colores/símbolos del mapa; activa `show_path/show_heat` si has usado esos parámetros en `render/print`, para que la leyenda los incluya también.

2.6. Clase Robot

`Robot(world, *, start=None)`

Si no se indica `start`, se coloca en una celda libre al azar (`world.random_free_cell()`). Lanza `ValueError` si `start` no es una celda libre.

<code>x, y, pos</code>	posición actual (<code>pos</code> es la tupla (x,y))
<code>wall_contact() ->{str: bool}</code>	{ <code>"N":...</code> , <code>"S":...</code> , <code>"E":...</code> , <code>"W":...</code> }; <code>True</code> si hay muro justo en esa dirección
<code>detect_objects(*, radius=0) ->List[str]</code>	objetos en la celda actual (o en un cuadrado de radio <code>radius</code> alrededor)
<code>move(direction) ->bool</code>	intenta moverse; <code>True</code> si tuvo éxito, <code>False</code> si colisiona (el robot no se mueve)

`move` acepta `"N"`, `"S"`, `"E"`, `"W"` y `"STAY"` (que siempre devuelve `True` sin desplazar al robot); cualquier otro valor lanza `ValueError`.

Importante — dos niveles de API. Los métodos de `HouseWorld` de la Sección 2.5.3 (`is_free`, `neighbors`, ...) son de *consulta*: no mueven nada, sirven para razonar sobre el mundo. Los de `Robot` (sobre todo `move`) son de *ejecución*: actúan de verdad y pueden fallar por colisión. En las Prácticas 1 y 2 (búsqueda/lógica) construirás un modelo o una base de conocimiento con los métodos de consulta; solo al final ejecutarás el plan encontrado con `Robot.move`. Esta práctica usa ambos niveles para que la distinción quede clara desde el principio.

2.7. Ejemplo de uso conjunto

```
from house_world import HouseWorld, Robot
```

```
world = HouseWorld(20, 14, num_rooms=5, seed=42)
robot = Robot(world)                                # posicion inicial aleatoria

if world.is_free(robot.x + 1, robot.y):             # consulta: se puede ir al este?
    robot.move("E")                                  # ejecucion: muevete de verdad

world.print(robot_pos=robot.pos)                    # visualiza el estado actual
```

3. Desarrollo de la práctica

Todas las funciones que se piden usan *solo* la API de la Sección 2: no hay que tocar `HouseWorld` ni `Robot`.

3.1. Paso 1: Ejecución inicial

Ejecuta el fichero tal cual se proporciona:

```
python3 practica0_enunciado.py
```

Debería generarse una casa y mostrarse con el robot colocado en ella (nada de lo que hay que implementar se llama todavía). Prueba también a cambiar la semilla (`--seed 1`, `--seed 2`, ...) para ver casas distintas, y a ejecutar directamente `common/house_world.py` con `--demo-path`, `--demo-heat` y `--cell-width/--cell-height` para ver esas capas de visualización antes de necesitarlas de verdad en las Prácticas 1 y 4.

3.2. Paso 2: `count_free_cells`

Implementa:

```
count_free_cells(world) ->int
```

Cuenta cuántas celdas de la casa son libres (no muro).

Descomenta en el `main` la línea correspondiente y comprueba el resultado.

3.3. Paso 3: `rooms_by_type`

Implementa:

```
rooms_by_type(world) ->Dict[RoomType, int]
```

Cuenta cuántas habitaciones hay de cada tipo. Solo se incluyen los tipos que aparecen al menos una vez (no debe haber entradas a 0).

3.4. Paso 4: `find_object`

Implementa:

```
find_object(world, name) ->Optional[(int,int)]
```

Devuelve la celda del primer objeto llamado `name` (según el orden de `world.all_objects()`), o `None` si no existe ninguno con ese nombre.

3.5. Paso 5: `nearby_walls`

Implementa:

```
nearby_walls(robot) ->List[str]
```

Devuelve, en orden alfabético, las direcciones ("N", "S", "E", "W") en las que el robot tiene un muro justo al lado.

3.6. Paso 6: `step_towards`

Implementa:

```
step_towards(robot, goal) ->Optional[str]
```

Mueve al robot **un** paso hacia `goal`, sin planificar: de entre los vecinos libres de su posición actual, elige el que minimiza la distancia Manhattan a `goal` (si hay empate, el primero según el orden N, S, E, W) y ejecuta `robot.move` en esa dirección. Devuelve la acción ejecutada, o `None` si el robot ya está en `goal` (y por tanto no se mueve).

Pregunta (no puntuable, para pensar): prueba a llamar `step_towards` repetidamente hacia un objetivo que esté en *otra habitación*. ¿Llega siempre al destino? ¿Qué le pasa cuando el camino más corto en línea recta choca con una pared? Guarda la respuesta: la Práctica 1 trata exactamente de resolver este problema como es debido.

3.7. Paso 7: `execute_moves`

Implementa:

```
execute_moves(robot, directions) ->List[(int,int)]
```

Ejecuta la secuencia de acciones `directions` sobre el robot, una a una, con `robot.move`. Devuelve la traza de posiciones *reales* del robot tras cada intento: si una acción colisiona, el robot no se mueve y esa misma posición se repite en la traza (no se lanza ninguna excepción).

4. Entregables

- Un único fichero `practica0_enunciado.py` completado, renombrado a `practica0.py`, entregado en la tarea del aula virtual habilitada para ello.
- El código debe ser legible y estar debidamente tabulado. No es necesario comentarlo explicando su funcionalidad.